Gráfica Multimedia

Producción

Lourdes Marco · lmarco@esdmadrid.es

ESDM

JavaScript

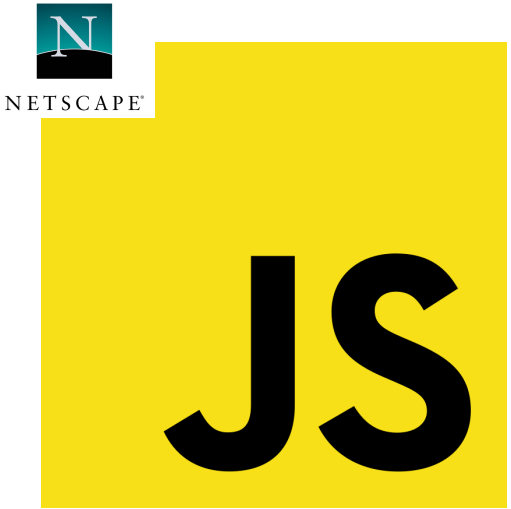
Más allá de la navegación

JavaScript

JavaScript (abreviado comúnmente **JS**) es un **lenguaje de programación interpretado**. (No necesita compilación, es un intérprete quien los ejecuta. Python y PHP también lo son).

Fue desarrollado originalmente por Brendan Eich (desarrollador en Netscape) con el nombre de Mocha, el cual fue renombrado posteriormente a LiveScript, para finalmente quedar como JavaScript.

JS **fue lanzado en 1995** con una actualización del navegador Netscape.



ECMAScript

JS sigue las directrices del lenguaje estándar **ECMAScript**, que se encarga de **regir cómo debe ser interpretado y cómo debe funcionar** el lenguaje JavaScript.

Las versiones de ECMAScript eran un poco lío, de modo que se llegó a determinación de nombrarlas por año, usando “ESX”, y siendo X el año de publicación.

Ahora mismo se está trabajando en la ES2027

https://www.w3schools.com/js/js_2027.asp



Anticipated Features to ES2027

SHARE

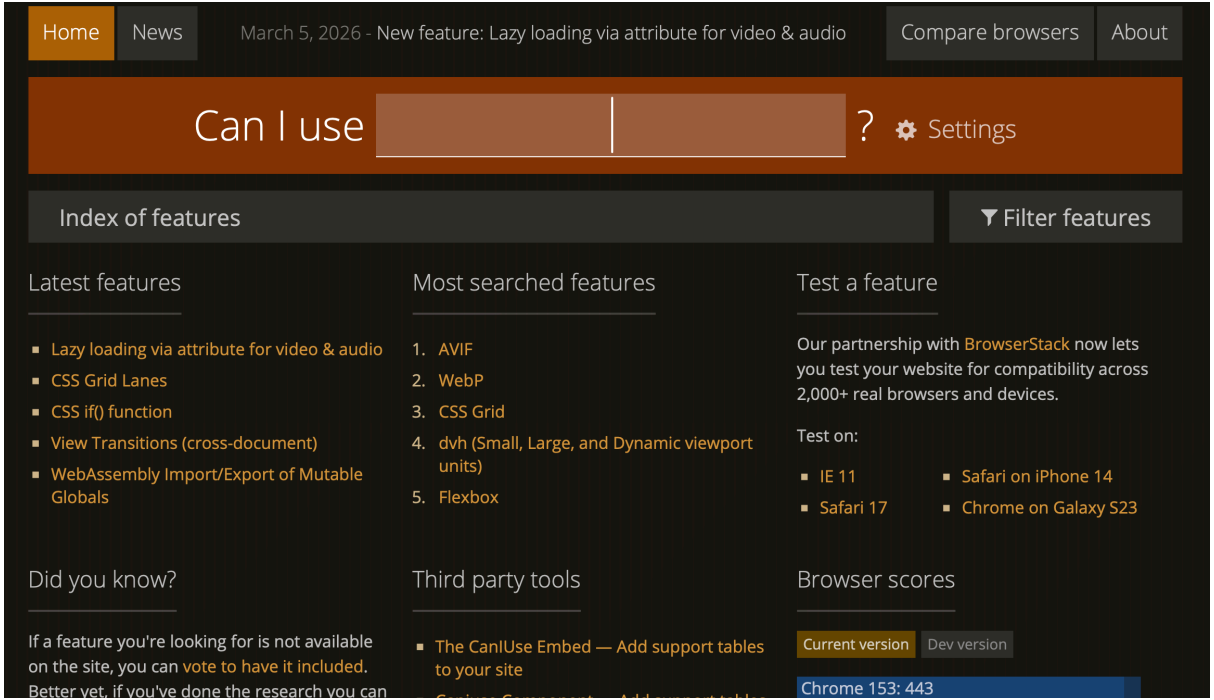
- **Temporal API**
  The long-time-overdue successor to the buggy, 30-year-old Date object. Unlike Date, Temporal objects are immutable and provide first-class support for time zones and non-Gregorian calendars.

- **Explicit Resource Management (using keyword)**
  C#-style block-scoped disposal variables that auto-clean up database connections and streams when leaving context, avoiding try...finally boilerplate.

Can I use (comprobar compatibilidad con navegadores)

Can I use es una herramienta muy interesante, porque permite comprobar la compatibilidad que tienen los navegadores con las funcionalidades.
Permite checar HTML, CSS y JS.

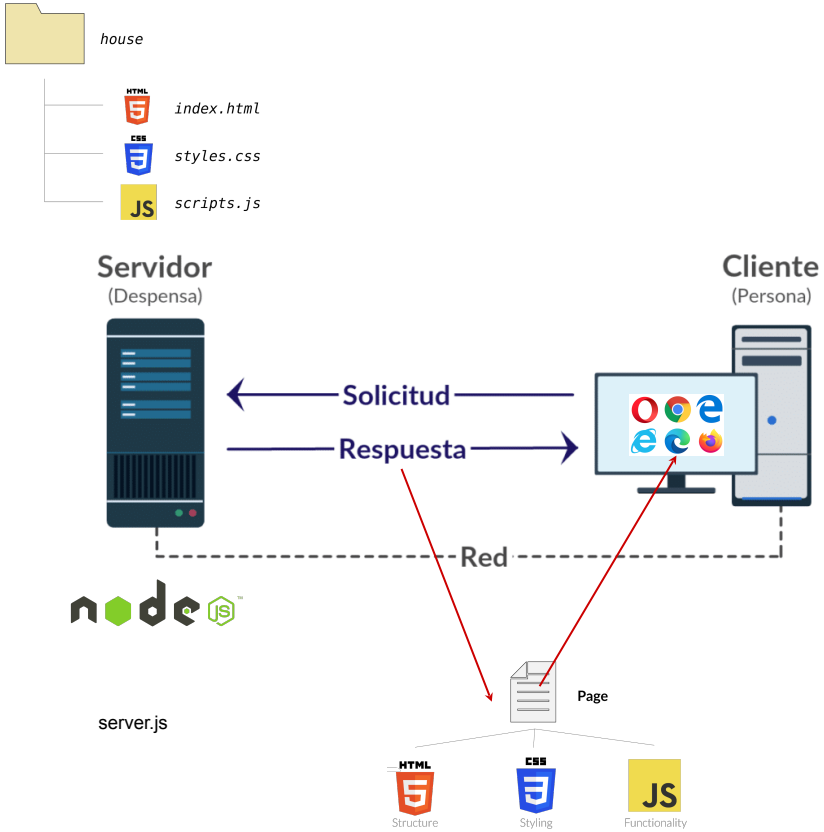
https://caniuse.com/



JS Cliente - JS Servidor

JavaScript se utiliza principalmente:

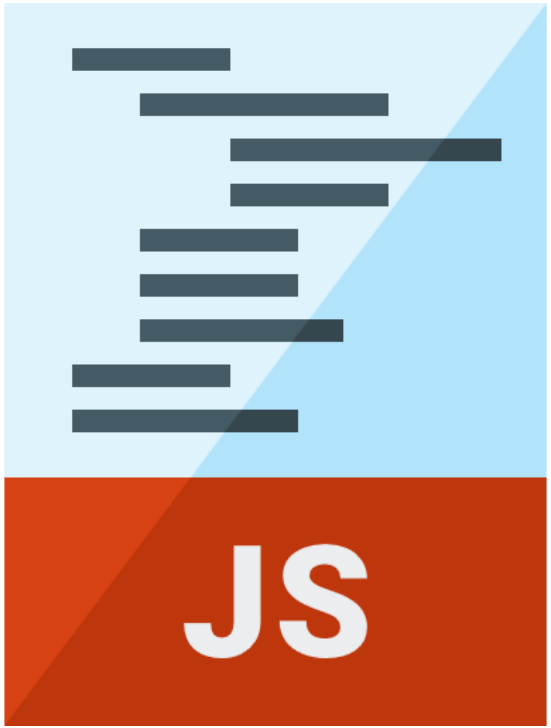
- **Del lado del cliente**. Implementado como parte de un navegador web permitiendo mejoras en la interfaz de usuario y páginas web dinámicas.
- **Del lado del servidor**. Instalando en el servidor herramientas que permiten el uso del lenguaje JavaScript para lanzar un servidor web y programar el servicio.



JS en el navegador

Los navegadores ejecutan el lenguaje JS de dos maneras diferentes:

- **Sentencias paso a paso** en la consola
- **Script JS** en una página HTML

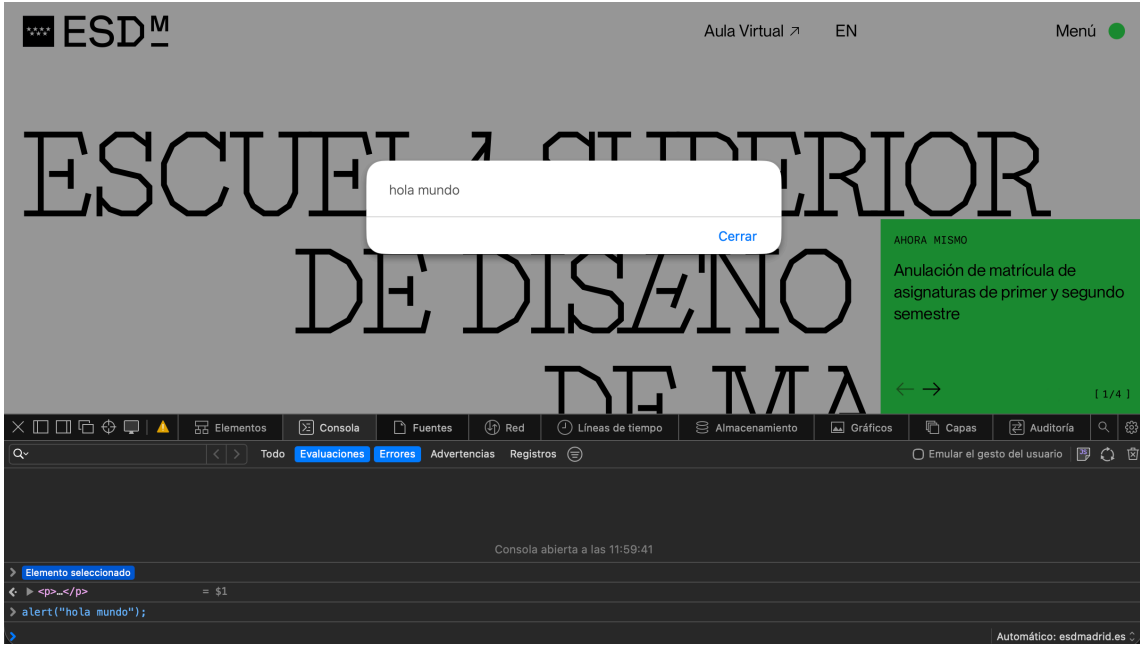


Sentencias JS paso a paso

Si hacemos **botón derecho -> inspeccionar** en cualquier parte del viewport, se abre el inspector de código, donde encontramos una pestaña llamada **“Consola”**.

Aquí podemos escribir sentencias JS de cualquier tipo, como sumas, restas, multiplicaciones… u otras más complejas que iremos aprendiendo.

Vamos a lanzar nuestra primera alerta escribiendo **alert(“Hola Mundo”);**
y pulsando el intro.

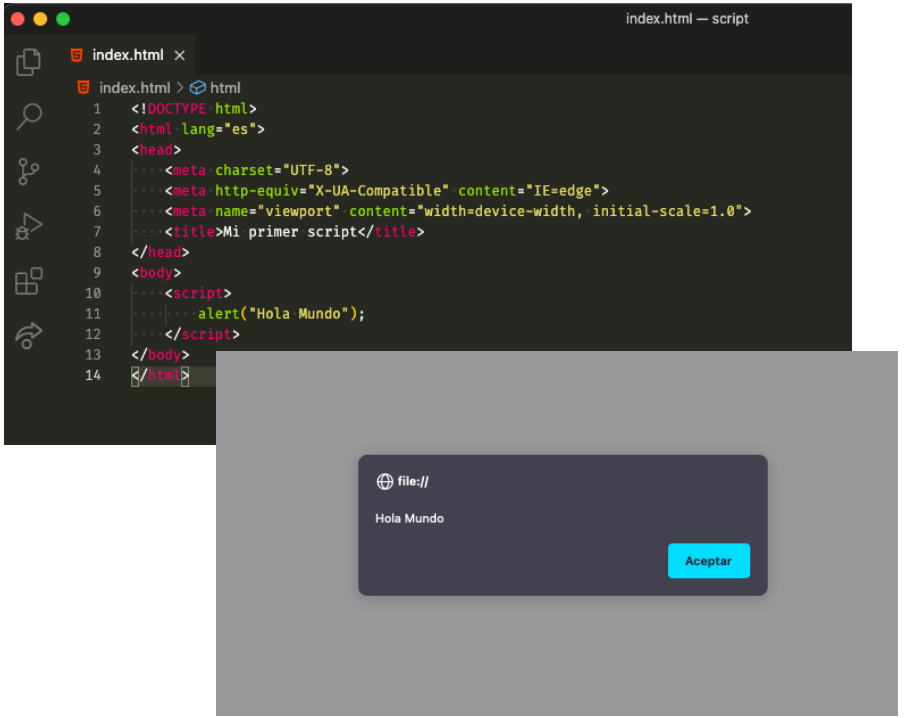


JS en el navegador | <script>

Un script es un programa o código escrito en lenguaje JS. Para incluir un script en un fichero HTML, podemos hacerlo usando la etiqueta <script>.
Usaremos de nuevo la alerta para esta prueba.

HTML

```
<body>
        <script>
                alert("Hola Mundo");
        </script>
</body>
```



JS en el navegador | fichero externo .js

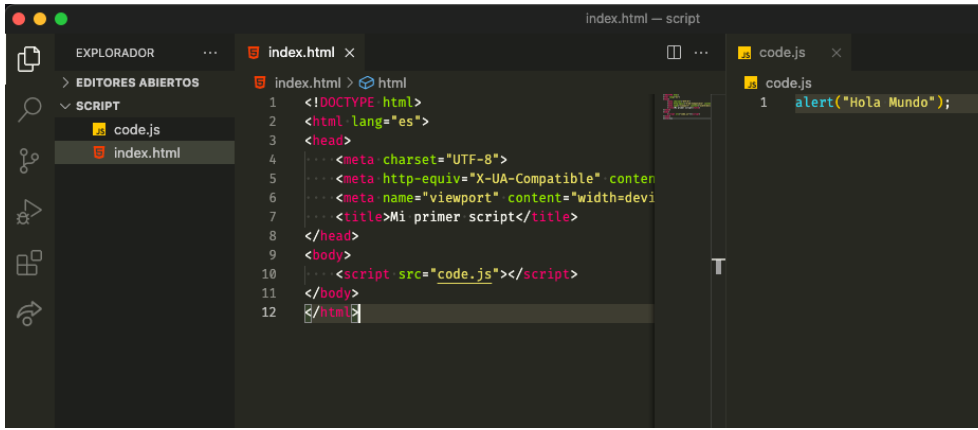
Igual que sucede con las hojas de estilo CSS, un programa o script JS también puede estar en un fichero externo. Es la forma más común de insertar código JS en las páginas HTML.

HTML

```
...
<body>
        <script src=”code.js”></script>
</body>
...
```

JS

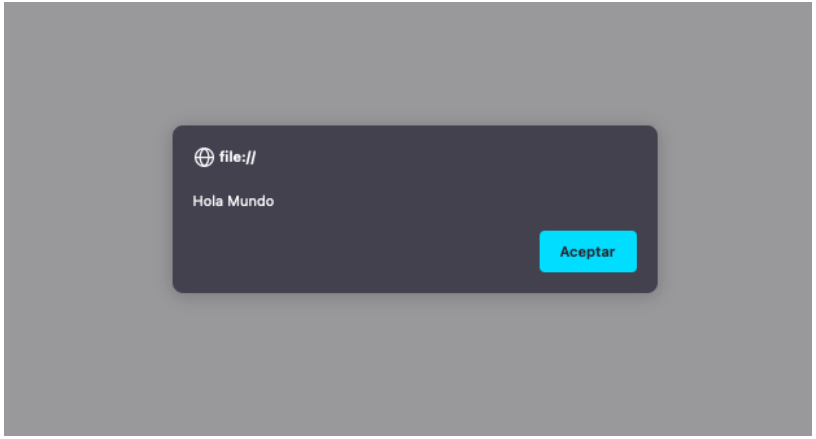
```
alert(“hola”);
```



alert(message)

Parameter	Type	Description
message	String	Optional. Specifies the text to display in the alert box, or an object converted into a string and displayed

alert() es un método o **función** que muestra un cuadro de alerta con un mensaje definido por el programador y un botón “Aceptar” para que el usuario confirme su lectura.



JS

let mensaje = “Hola Mundo”;
alert(mensaje);

Programa, sentencia, código fuente

- **Sentencia / declaración**. Es una **orden al procesador.** Con ella describimos una **tarea a realizar.** Todas las sentencias acaban en; (punto y coma)

- **Programa**. **Secuencia de sentencias** que se ejecutan en el orden establecido.

- **Código fuente**. Se refiere al **texto con las sentencias y comentarios de un programa**.

- **Comentarios**. **Texto con valor informativo**. Sirven para documentar el código.

JS

```
/*
        Código fuente de un
    Programa básico en JS
*/

// Primera sentencia
alert(“hola mundo!”);

// Segunda sentencia
alert(“:)”);
```

Variables

- Una variable es un **contenedor de valores**, cuyo contenido puede variar a lo largo de un programa.
- Las variables de JavaScript son **no tipadas**, es decir, pueden contener cualquier tipo de dato.

JS

```
/*
        Hemos guardado un número
        y una palabra, pero la forma
        de declarar las variables es
        la misma
*/

let num = 10;
let palabra = “Hola”;
```

Declarar variables y cambiar su valor

Declarar variables es indicar al programa que ese espacio de memoria va a ser usado con ese nombre. Para declarar variables usamos let y const

Podemos cambiar el valor de una variable tipo let a lo largo de un programa sin necesidad de volver a declararla. Sin embargo no podemos cambiar el valor de una variable tipo const, porque es una constante.

JS

```
/*
            Hemos declarado dos variables
            una que cambiará y otra que no
*/
let num = 10;
const num2 = 10;

/* aquí estamos cambiando el valor
            de la variable num */

num = num + 5;

/* error */
num2 = 20;
```

Normas para declarar variables

Las variables no pueden tener cualquier nombre.

Existen unas normas:

- El **primer carácter debe ser una letra**, **guión bajo** (_) o signo de **dólar** ($).

- Los nombres **NO pueden incluir**
  - números
  - operadores matemáticos o lógicos
  - símbolos especiales (%, # ...)
  - espacios

JS

```
let $nombre = “Pepe”;
let _nombre = “Pepa”;
let nombre;
```

JS

```
let 2nombre = “Pepe”;
let mi nom = “Pepa”;
let nom+apell;
let nom-apell = “Pepe Apellido”;
```

Palabras reservadas

Hay palabras reservadas que tampoco pueden usarse para declarar variables, porque **pertenecen al propio lenguaje** y se usan para otros fines.

Las palabras reservadas de ECMAScript 6 son:

```
break  case  catch  class  const  continue  debugger  default  delete  do
else  export  extends  finally  for  function  if  import  in  instanceof
new  of  return  super  switch  this  throw  try  typeof  var  void  while
with  yield
```

Tampoco podemos utilizar como identificadores los **literales**:

```
null  true  false
```

También tenemos **palabras reservadas que se publicaron en versiones anteriores de ECMAScript**, que finalmente no han sido utilizadas, pero que no se deben utilizar:

```
abstract  boolean  byte  char  double  final  float  goto  int  long  native
short  synchronized  throws  transient  volatile
```

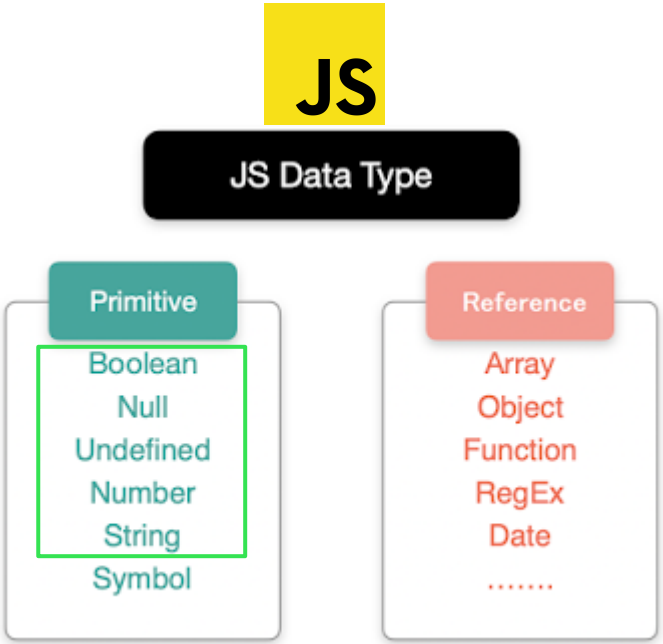
Tipos de datos

Tipo de dato se refiere a los **diferentes valores con los que un programa** JS **puede operar**.

```
4   typeof("Hola") // string
5   typeof(12) // integer
6   typeof(true) // boolean
7   typeof(undefined) // undefined
8   typeof(null) // object
9   typeof({}) // object
10  typeof(Symbol()) // symbol
11  typeof(1n) // bigint
12  typeof(function(){}) // function
```



Tipos de datos JS



JS

```
//numbers
let x = 10;
let y = 2.5;

//strings
let nombre = “Marie”;
let apellido = ‘Curie’;

//undefined
let x;
let y;

//boolean
let isHappy = true;
let isHoliday = false;

//null
let x = null;
```